Programmable Switch Telemetry at Microsecond Granularity: BurstMon in Action

Yinchuan Cong[1], Kun Xie[1,], Jiajun He[1], Jingyang Lin[1], Xin Zeng[1], Lele Wang[1], Jigang Wen[2], Kenli Li[1], Shiming He[3], Wei Liang[2], Gaogang Xie[4]*

[1]Hunan University [2]Hunan University of Science and Technology [3]Changsha University of Science and Technology [4]CNIC CAS; UCAS, China

Abstract

Modern high-speed networks exhibit highly dynamic traffic behaviors, where microsecond-scale bursts and rate shifts can significantly impact congestion control, scheduling, and anomaly response. Most existing telemetry systems operate at millisecond granularity, missing critical fine-grained events. Recent work such as μMon pushes monitoring resolution toward microseconds by compressing rate traces with wavelet transforms, but its multi-timeslot buffering introduces milliseconds of latency that fundamentally limits responsiveness.

We make a complementary observation about the structure of microsecond-scale flow signals: they are sparse in the **event domain**—most flows exhibit long stable periods punctuated by a small number of sharp transitions (i.e., rising and falling edges). This insight motivates **BurstMon**, a microsecond-resolution telemetry system that detects change points directly in the data plane and reconstructs per-flow rate curves via sparse event reporting and control-plane interpolation.

BurstMon introduces two key advances over prior rotating-sketch systems: (1) a *time-sketch* with dual-purpose per-cell timestamps that unify logical lazy clearing and transition-driven chi-square triggering, eliminating dedicated cleaning sketches; and (2) a complete event-domain telemetry pipeline—from in-switch change-point detection via logarithmic-projection-based chi-square testing, through minimal digest reporting, to control-plane curve reconstruction. We provide formal guarantees on false-positive rate and stable-period reconstruction error.

Implemented on an Intel Tofino switch and evaluated on four production-inspired workloads, BurstMon achieves over 95% cosine similarity at 10–80 μs resolution, maintains control-plane bandwidth below 0.07 Gbps, and delivers response latency of $\sim$410 μs—much faster than μMon. We demonstrate its value through congestion attribution, pulse-wave DDoS detection, elephant flow steering, and proactive congestion signaling.

1 Introduction

Network traffic in modern data centers and backbone networks is increasingly volatile: microbursts, transient rate surges, and sub-millisecond congestion episodes arise and dissipate within a few microseconds [14, 19, 29]. These fine-grained dynamics have direct consequences for performance-critical operations. In feedback-driven protocols such as ECN [1, 18], for example, switches mark packets during bursts, triggering upstream rate adjustments that unfold over hundreds of microseconds—well below the observability of conventional monitoring tools. As link speeds reach 100 Gbps and beyond [29], the gap between the timescale of network events and the timescale of monitoring has become a first-order obstacle to effective congestion control, anomaly detection, and adaptive scheduling [21, 27, 28].

Unfortunately, most deployed telemetry systems operate at millisecond or coarser granularity. Sampling-based approaches (e.g., NetFlow [8], sFlow [25]) and sketch-based aggregation methods (e.g., Elastic Sketch [30], NZE Sketch [15], CocoSketch [31]) are designed for long-term trend analysis and cannot capture the sub-millisecond dynamics that govern latency-sensitive workloads. Figure 1 illustrates this gap concretely: the fine-grained curve (10 μs, red) reveals burst patterns, stepwise rate changes, and flow start/stop behavior that are entirely smoothed out at 1 ms resolution.

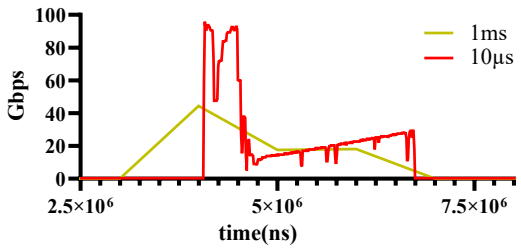


Figure 1: Comparison of per-flow rate measurements at 1 ms vs. 10 μs granularity. The fine-grained curve reveals dynamics invisible at coarse resolution.

Recent in-data-plane systems such as ConQuest [7],

Snappy [6], and BurstRadar [16] have made progress toward sub-millisecond visibility using a rotating *Count-Min Sketch (CMS)* snapshot mechanism to identify flows responsible for microburst-induced queue buildup. However, these systems share a common scope—they perform local, per-packet decisions (marking, dropping, or rerouting) without exporting per-flow rate histories for control-plane reconstruction or cross-flow analysis. This distinguishes BurstMon from these prior systems: while ConQuest and Snappy answer *which flows are heavy right now?* via in-switch thresholding, BurstMon answers a fundamentally different question—*what does each flow's rate curve look like over the past milliseconds?*—enabling downstream analytics such as congestion attribution, pulse-wave DDoS detection, and proactive congestion signaling that require full temporal context, not just instantaneous snapshots.

Achieving this goal is non-trivial. Programmable switches [17, 35] impose severe constraints: no floating-point arithmetic, no loops, limited per-packet computation, a fixed number of pipeline stages (e.g., 12 ingress stages on Tofino1), and strict memory bounds (e.g., ∼120 MB SRAM [34]). Moreover, the data-to-control-plane uplink is a severe bottleneck: uploading a 128 KB sketch every 10 μs would require 102.4 Gbps—beyond most current infrastructure capabilities. The state-of-the-art approach, μMon [33], compresses sketch data using Haar wavelet transforms, but wavelet compression requires accumulating more than a hundred consecutive timeslots before compression can begin, introducing over 1 ms of buffering latency that fundamentally limits responsiveness.

A different signal model. We observe that microsecond-scale flow rate curves are sparse not in the frequency domain, but in the **event domain**: most flows exhibit long stable periods punctuated by a small number of sharp transitions (i.e., rising and falling edges)—bursts. This shift in perspective—from *compressing measurements* to *detecting and reporting events*—enables a fundamentally different telemetry architecture. Rather than uploading compressed sketches periodically, we detect statistically significant rate changes in the data plane and report only these sparse change points to the control plane, which reconstructs full per-flow rate curves via interpolation.

Based on this insight, we present **BurstMon**, a lightweight monitoring framework that exploits event-domain sparsity to achieve microsecond-resolution per-flow visibility. BurstMon detects statistically significant rate changes in the data plane using a chi-square test on first-order rate differences and transmits only these sparse, high-signal change points to the control plane. By transmitting events rather than compressed sketches, BurstMon achieves accurate reconstruction with minimal memory and bandwidth overhead—and with end-to-end response latency of approximately 410 μs, over an order of magnitude faster than wavelet-based alternatives. Code is available at https://github.com/YinchuanCong/BurstMon_tofino.git.

Contributions

- **Event-domain sparsity as a design principle for microsecond telemetry.** We identify that microsecond-scale flow rate signals are compressible in the event domain—not the frequency domain—and build a telemetry architecture around this observation. BurstMon detects statistically significant rate deviations using first-order differences and a chi-square test, reporting only sparse change points from the data plane. We provide a formal false-positive bound (Proposition 1) and characterize the maximum unrecorded rate deviation during stable periods (Equation 11), establishing theoretical guarantees on both detection reliability and reconstruction fidelity.
- **A time-sketch data structure with dual-purpose timestamps.** Building on the rotating snapshot mechanism of ConQuest [7] and Snappy [6], we design a time-sketch that augments each CMS cell with a per-cell timestamp. This timestamp serves two tightly coupled purposes at zero additional cost: it enables *logical lazy clearing* of stale data without bulk register resets, and it provides *transition-driven triggering* of the chi-square test—unifying memory management and statistical computation into a single comparison on the update path. A three-sketch rotation scheme ensures strict temporal isolation across consecutive timeslots.
- **An efficient data-plane chi-square test via logarithmic projection.** To approximate multiplication, division, and square root under strict hardware constraints, we introduce a logarithmic projection method that reduces the burst score computation to 9 table lookups across 7 pipeline stages. At scaling factor $k = 512$, the effective threshold is confirmed empirically with MSE < 0.01 and accuracy $> 99.5\%$ on Tofino.
- **Prototype implementation, four use cases, and comprehensive evaluation.** We implement BurstMon on Intel Tofino1 and evaluate it on four workloads with diverse traffic characteristics (Crest Factors 1.13–2.62). Under 10–80 μs sampling windows, BurstMon achieves over 95% cosine similarity, maintains control-plane bandwidth below 0.07 Gbps, and reduces response latency by over 10× compared to μMon. We demonstrate four practical applications: congestion attribution (macro-F1 of 0.938 vs. 0.619 for μMon), pulse-wave DDoS detection, elephant flow steering (12% P99 reduction), and proactive congestion signaling (16× faster than ConQuest).

2 Key Insight: Event-Domain Sparsity

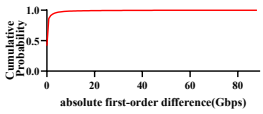
Despite the promise of programmable switches, achieving microsecond-level per-flow monitoring remains a challenging systems problem due to three constraints. First, *high data volume*: mirroring a 40 Gbps link at microsecond resolution

produces up to 5 GB/s of raw traffic—far exceeding switch-to-controller bandwidth (10 Gbps in digest). Second, *sketch export limits*: achieving low sketch error at high frequency requires large tables and frequent exports, increasing control-plane load. Third, *compute limitations*: traditional compression algorithms are too expensive for line-rate execution, and wavelet-based approaches [33] incur multi-millisecond buffering latency.

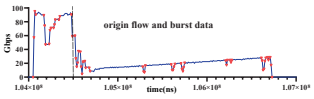
Prior work μMon [33] on microsecond-scale telemetry employs wavelet-based compression to reduce the volume of rate measurements uploaded to the control plane. We make a complementary observation about the structure of microsecond-scale flow signals: they are sparse not only in the frequency domain, but—more directly—in the **event domain**: most flows exhibit long stable periods punctuated by a small number of sharp transitions (bursts). This event-level sparsity motivates a fundamentally different system architecture: instead of compressing and transmitting full rate traces, we detect and report only the burst events themselves, together with lightweight metadata sufficient for the control plane to reconstruct accurate per-flow rate curves.

To validate this insight, we analyze production-scale traffic from the WebSearch25 dataset at 10 μs resolution. We compute the absolute first-order rate differences of a flow across consecutive timeslots; the resulting CDF (Figure 2a) reveals strong sparsity: 90% of absolute differences of that flow remain below 3.192 Gbps. The majority of significant variations are concentrated in a small number of rising and falling edges—we characterize these sparse yet significant changes as *bursts*.

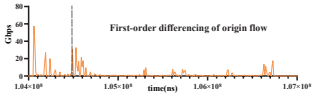
Using these change points as the sole reporting events, Figure 2d shows that linear interpolation over sparse burst points achieves a cosine similarity of 0.99 with the ground truth, validating that event-based reporting yields high-fidelity rate curves with minimal overhead.



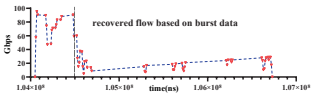
(a) CDF of absolute first-order differences.



(b) Original flow with detected bursts.



(c) Absolute first-order differences.



(d) Reconstructed curve via interpolation.

Figure 2: Flow rate analysis and reconstruction at 10 μs granularity on WebSearch25.

Motivated by this insight, BurstMon's design follows three core principles: (i) leverage first-order differences to amplify rate changes and enable sensitive detection; (ii) suppress stable periods and report only statistically significant bursts; and (iii) reconstruct full flow curves via interpolation with minimal communication overhead.

Realizing this design on programmable hardware introduces four engineering challenges, each addressed by a dedicated component of BurstMon:

Q1: How to support accurate per-flow measurement? Maintain current and previous flow states and compute rate deltas efficiently using compact, parallel-friendly data structures (Section 4.1).

Q2: How to detect bursts under strict resource constraints? Ensure detection logic fits within switch pipeline limitations and executes at line rate (Section 4.2).

Q3: How to approximate arithmetic under hardware constraints? Design low-error, low-memory approximations for multiplication, division, and square root without floating-point support (Section 4.3).

Q4: How to reconstruct flow curves for analysis? Enable the control plane to reconstruct continuous flow behavior from sparse burst reports (Section 4.4).

3 Related Work

We organize related work into three categories that collectively define the gap BurstMon addresses.

In-data-plane measurement systems. Programmable switches have enabled *in-network monitoring* (INM) [35], where measurement logic executes at line rate in the data plane. Snappy [6] and ConQuest [7] maintain multiple rotating CMS snapshots to track per-flow queue occupancy, enabling microsecond identification of heavy hitters responsible for microburst-induced queue buildup. Building on a similar differential sketch idea, δ-sketches [12] compare ingress and egress CMS instances for flow-level loss detection. However, they focus on local, per-packet decisions and do not export fine-grained per-flow rate histories for offline reconstruction or cross-flow analysis. Table 1 in Section 4.1.5 provides a detailed comparison along five dimensions.

Sketch compression and export. At microsecond resolution, the communication bandwidth between data and control planes becomes a severe bottleneck. Prior work [22, 26, 31] leverages sketch-based methods to summarize traffic, allowing the control plane to periodically reconstruct flow-level statistics. Compression-based techniques—NZE Sketch [15], CS-Sketch [20], LightGuardian [32], and BitSense [11]—mitigate upload overhead but still fall short under high-frequency reporting. μMon [33] applies Haar wavelet transforms to compress sketch data, achieving high accuracy but requiring 128–512 consecutive timeslots of buffering (1.28–5.12 ms at 10 μs resolution), which fundamentally limits its responsiveness.

Coarse congestion signals. Other approaches rely on aggregate congestion indicators such as ECN marking [18] or BurstRadar's queue-level burst flags [16]. While useful for triggering immediate reactions, these signals lack per-flow fidelity and cannot support temporal analytics such as congestion attribution or pulse-wave DDoS detection.

The problem we address—*reconstructing per-flow rate curves at microsecond resolution from sparse reports under a limited uplink bandwidth budget*—requires fundamentally different design choices, including determining which rate changes are statistically significant, bounding the false-positive rate of such decisions, and reconstructing continuous temporal profiles from sparse events.

4 System Design

BurstMon is a programmable switch-based monitoring system that performs lightweight, real-time burst detection in the data plane and reconstructs per-flow behavior in the control plane. Figure 3 illustrates the overall architecture: a high-speed data plane performs burst detection and reporting, while a control plane reconstructs flow-level dynamics and supports downstream analytics. We describe each component in the order of the four challenges identified in Section 2.

Throughout this section, we let r_i denote the traffic volume of a monitored flow in the i-th timeslot, and define the first-order rate difference $a_i = |r_i - r_{i-1}|$ and the cumulative variation $s_i = \sum_{k=1}^{i} |r_k - r_{k-1}|$. Since the timeslot duration Δt is fixed, volume r_i and rate $r_i/\Delta t$ differ only by a constant factor. We use r_i (volume) for notational simplicity and refer to it as "rate" where context is clear. The data plane maintains three per-flow quantities: r_i, r_{i-1}, and s_i, from which the burst detection logic operates.

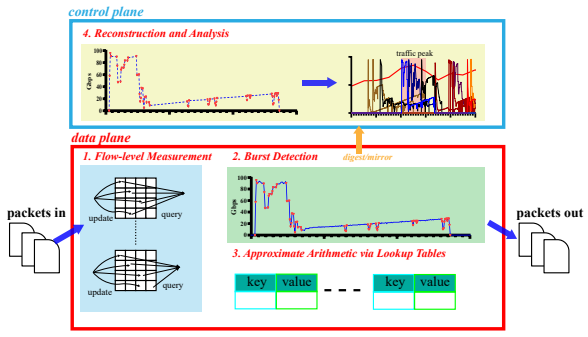


Figure 3: System architecture of BurstMon. The design spans both data and control planes and addresses four core challenges (Q1–Q4).

4.1 Per-flow Measurement via Time-Sketch (Q1)

4.1.1 Limitations of the Dual-CMS Approach

A straightforward approach to maintain r_i and r_{i-1} is to alternate between two CMS instances, with one holding the current timeslot's data and the other storing the previous slot's state. While conceptually simple, this scheme is difficult to implement correctly on programmable switches. The P4 programming model lacks support for loops, recursion, or full register traversal, so the system cannot explicitly clear all CMS counters at the beginning of a new timeslot. Without the ability to reset outdated counters, stale values from the $(i-2)$-th timeslot persist. Under hash collisions, these residuals contaminate the estimate of r_i, making the dual-CMS model unsuitable for high-frequency monitoring.

4.1.2 Time-Sketch Design

To overcome this limitation, BurstMon introduces a data structure called *time-sketch*, which extends the rotating snapshot mechanism of ConQuest [7] and Snappy [6] with per-cell timestamps for logical clearing and a three-sketch rotation scheme.

The time-sketch extends the classic CMS by associating a *timestamp* with each counter cell. As shown in Figure 4, each cell (i, j) of a $d \times w$ sketch contains two fields: $value^{i,j}$, the accumulated traffic volume, and $t^{i,j}$, the most recent timeslot in which the cell was updated.

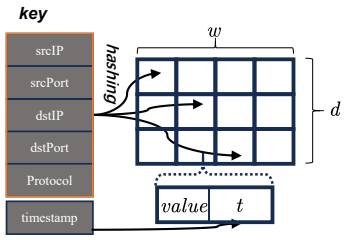


Figure 4: Time-sketch structure. Each CMS cell maintains a timestamp field to support logical clearing of stale data.

For an incoming packet p with key key, length L_p, and current timeslot t_p, the update procedure at each cell (i, j) with $j = h_i(key)$ is:

- If $t^{i,j} \neq t_p$ (stale cell): logically clear by setting $value^{i,j} \leftarrow L_p$ and $t^{i,j} \leftarrow t_p$;
- If $t^{i,j} = t_p$ (current cell): increment $value^{i,j} \leftarrow value^{i,j} + L_p$.

Query operations follow standard CMS semantics: the minimum value across d hash rows is returned.

4.1.3 Three-Sketch Rotation Mechanism

To achieve strict timeslot isolation, BurstMon maintains three time-sketch instances—Tsk_0, Tsk_1, and Tsk_2—rotated across timeslots. Let i denote the current timeslot index. Upon receiving a packet in timeslot $t_p = i + 1$, the system performs:

1. Query $Tsk_{(i-1) \bmod 3}$ to retrieve r_{i-1};
2. Query $Tsk_{i \bmod 3}$ to retrieve r_i;

3. Compute $a_i = |r_i - r_{i-1}|$ and insert a_i into CM_s (the cumulative-variation sketch);
4. Update $Tsk_{(i+1) \bmod 3}$ with the new packet using timestamp-aware logic.

Figure 5 illustrates this rotation. At each step, only the sketch from two timeslots ago is overwritten; logical clearing ensures no residual data contaminates the current timeslot.

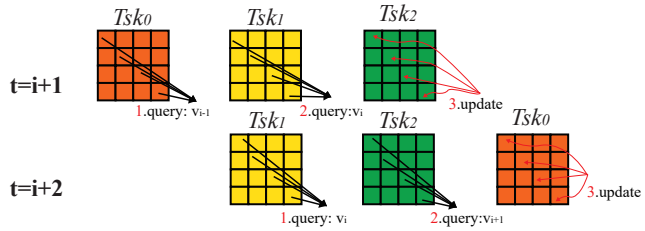


Figure 5: 3-sketch rotation in BurstMon: each time-sketch instance is reused every third timeslot, ensuring clean temporal separation.

Worked example. Consider a sketch with $d = 2$ rows (hash functions h_0, h_1) and width $w = 4$, and suppose the current timeslot just advanced to $i+1 = 6$. A packet of length $L_p = 1500$ B arrives for flow f with $h_0(f) = 1$ and $h_1(f) = 3$.

Step 1 (Query Tsk_1 for r_{i-1}). Cells $(0,1)$ and $(1,3)$ of Tsk_1 store values 4500 and 6000, both with timestamp 4. The CMS estimate is $r_{i-1} = \min(4500, 6000) = 4500$ B.

Step 2 (Query Tsk_2 for r_i). Cells $(0,1)$ and $(1,3)$ of Tsk_2 hold values 7500 and 7500 with timestamp 5. Thus $r_i = 7500$ B.

Step 3 (Compute difference and update CM_s). $a_i = |7500 - 4500| = 3000$ B, inserted into the cumulative-variation sketch.

Step 4 (Update Tsk_0 with the new packet). At cell $(0,1)$ of Tsk_0, the stored timestamp is 3 (stale), so the cell is logically cleared: $value^{0,1} \leftarrow 1500$, $t^{0,1} \leftarrow 6$. At this point the χ^2 burst test is invoked with $a_i = 3000$, s_i, and i to decide whether a burst report should be emitted (Section 4.2).

A second packet (1000 B) arriving for f in the same timeslot finds $t^{0,1} = 6 = t_p$ and simply increments: $value^{0,1} \leftarrow 2500$ and this arrival does not trigger the χ^2 test.

4.1.4 The Timestamp as a Dual-Purpose Primitive

The per-cell timestamp serves two tightly coupled architectural purposes, both realized at $O(1)$ per-packet cost.

Purpose 1: Logical lazy clearing. Programmable switches do not support bulk register resets or runtime traversal of sketch memory. The timestamp sidesteps this limitation: a mismatch ($t_p \neq t^{i,j}$) indicates stale data and triggers an immediate overwrite, effectively a logical reset without any physical clearing operation. A match ($t_p = t^{i,j}$) results in a standard counter increment. Clearing is thus amortized over normal packet processing, eliminating dedicated clean-up stages.

Purpose 2: Transition-driven χ^2 triggering. Beyond memory management, the timestamp mismatch serves as a zero-overhead detector of timeslot boundaries. When an incoming packet is the **first** to cross into a new timeslot for a given flow (evidenced by $t_p \neq t^{i,j}$), BurstMon immediately triggers the χ^2 burst test on the just-completed measurement window. The χ^2 test therefore fires **at most once per flow per timeslot**, driven naturally by traffic arrival rather than by costly periodic scans.

This dual use distinguishes BurstMon from ConQuest-style rotation, where clearing and statistical operation(aggregation) are two independently scheduled concerns. The time-sketch unifies them: a single timestamp comparison simultaneously performs logical clearing and detects the exact moment to invoke the χ^2 test, turning what is ordinarily a hardware limitation (the inability to reset registers) into an architectural advantage.

4.1.5 Comparison with Related Sketch Systems

Table 1 summarizes how BurstMon differs from ConQuest [7], Snappy [6], and δ-sketch [12] across five dimensions. Snappy and ConQuest track queue occupancy to identify congestion-causing heavy hitters, while δ-sketch detects per-flow packet loss via spatial ingress–egress differencing. BurstMon instead computes a *temporal* first-order difference across adjacent time windows, feeding an in-switch χ^2 test for microsecond-level rate monitoring.

Regarding state clearing, Snappy and ConQuest dedicate at least one full sketch instance to a cleaning role; δ-sketch uses an auxiliary bitmask register. BurstMon's time-sketch eliminates dedicated cleaning entirely: a cell is simply overwritten when its timestamp reveals staleness, and all sketch instances remain continuously available for both updates and queries.

The combination of time-sketch and 3-sketch rotation provides several key properties: (a) no bulk register resets, bypassing P4's lack of loops and register traversal; (b) strict temporal isolation, as each sketch instance is logically bound to a specific timeslot and rewrite only after two others have elapsed; (c) resource efficiency, with each cell requiring only a 32-bit counter and a 16-bit timestamp; and (d) full P4 compatibility, as all operations execute without control-plane interaction or runtime loops.

4.2 Chi-Square Burst Detection (Q2)

BurstMon detects flow rate bursts in the data plane and reports only significant change points to the control plane. The detection logic must execute within the switch pipeline's limited stage depth and without floating-point support.

BurstMon uses a lightweight burst detection algorithm based on the **chi-square test** [5,13,24], adapted for data-plane execution. Compared to the Kolmogorov–Smirnov [9] or Anderson–Darling [4] tests, the chi-square test requires no dis-

Table 1: Comparison of BurstMon with related sketch-based monitoring systems.

System	Measurement Objective	Core Sketch Structure	Rotation Mechanism	State Clearing	Statistical Triggering
Snappy	Identify queue-occupant heavy hitters	CMS snapshots (per traffic window)	Round-Robin between snapshots	Active clearing via auxiliary sketch	External / per-packet
ConQuest	Fine-grained queue delay contribution	CMS snapshots (per time window)	Round-Robin (Read/Write/Clean)	Active clearing via auxiliary sketch	Auxiliary timer / per-packet check
δ-sketch	Per-flow packet loss detection	Δ-Sketch ($S_{in}-S_{out}$)	Reactive (queue-depth triggered)	Bitmask + recirculation	Queue-depth threshold
BurstMon	**Microsecond flow rate reconstruction**	**Time-Sketch (CMS + timestamp)**	**3-Sketch Rotation (timeslot isolation)**	**Logical lazy clearing (timestamp)**	**timestamp mismatch**

tributional assumptions and has relatively low computational overhead, making it well-suited for resource-constrained environments.

Statistical formulation. BurstMon treats the sequence of absolute rate differences $a_1, a_2, \ldots, a_i$ as observations drawn from a common distribution under the null hypothesis of *stationarity*: all timeslots share the same expected rate difference $\bar{a} = s_i/i$. To test whether the most recent observation a_i deviates significantly from this historical average, we partition the observations into two groups—*historical* (timeslots 1 through $i-1$, with aggregate $O_{\text{hist}} = s_i - a_i$) and *current* (timeslot i, with $O_{\text{curr}} = a_i$)—and apply a goodness-of-fit test with 1 degree of freedom:

$$\chi^2 = \sum \frac{(O-E)^2}{E} = \frac{\left(s_i - a_i - \frac{s_i}{i}(i-1)\right)^2}{\frac{s_i}{i}(i-1)} + \frac{\left(a_i - \frac{s_i}{i}\right)^2}{\frac{s_i}{i}} \quad (1)$$

where the expected values under the null are $E_{\text{hist}} = \frac{s_i}{i}(i-1)$ and $E_{\text{curr}} = \frac{s_i}{i}$. Since $s_i - a_i - \frac{s_i}{i}(i-1) = -(a_i - \frac{s_i}{i})$, the two terms collapse to:

$$\chi^2 = \left(a_i - \frac{s_i}{i}\right)^2 \cdot \frac{i^2}{s_i(i-1)} = \frac{(a_i \cdot i - s_i)^2}{s_i(i-1)} \quad (2)$$

Intuitively, the statistic measures how many "normalized standard deviations" the current rate change a_i lies away from the historical average $\bar{a} = s_i/i$. A small χ^2 value indicates consistency with prior patterns; a large value signals a statistically significant deviation—a burst event.

Normalized burst score. The χ^2 statistic in Equation 2 involves compound multiplications and divisions prone to integer overflow under constrained bit-widths. BurstMon reformulates it as a normalized **burst score** by taking the square root:

$$\text{score}(f,i) = \frac{|a_i \cdot i - s_i|}{\sqrt{s_i(i-1)}} \quad (3)$$

A higher score indicates a sharper deviation from historical behavior and thus a more likely burst.

Cold start handling. For new flows (where $s_i = 0$), BurstMon reports the flow directly to the control plane, ensuring accurate reconstruction from the first timeslot without division by zero.

Evaluating Equation 3 in the data plane remains challenging due to the lack of floating-point support and potential integer overflow. BurstMon overcomes these limitations with a lookup-based logarithmic projection technique, described next.

4.3 Approximate Arithmetic via Lookup Tables (Q3)

The burst score in Equation 3 cannot be directly evaluated in the data plane due to strict memory and computational limitations. Logarithmic projection—a classical technique for approximating arithmetic via lookup tables [3, 10]—offers a natural solution. Prior work has explored approximate arithmetic on programmable switches: NetFC [10] enables floating-point emulation via log-domain table lookups, and PINT [3] employs similar integer approximations for in-band telemetry encoding. BurstMon builds on these foundations but targets a more complex computation—the full chi-square burst score—which chains multiplication, subtraction, division, and square root into a single per-packet pipeline. This requires 9 coordinated lookup operations across 7 pipeline stages.

To reduce memory pressure, BurstMon first applies a downscaling step: both a_i and s_i are right-shifted by 16 bits before burst detection. This significantly lowers the operand range while preserving relative accuracy. If a burst is detected, the switch emits a `digest` containing the original 32-bit counter pair (r_i, r_{i-1}), flow ID, and timestamp for precise control-plane reconstruction.

Even with 16-bit operands, directly evaluating the burst score using match-action tables is infeasible. A naive approach using all three 16-bit quantities (a, s, i) as composite keys would require $2^{48} \approx 281$ trillion entries ($\sim$30 TB). A more compact approximation is necessary.

4.3.1 Logarithmic Projection

BurstMon adopts **logarithmic projection**, which approximates multiplication and division by mapping operands to their base-2 logarithms, performing addition or subtraction in the log domain, and exponentiating the result. Let $\tilde{x} = \lfloor k \log_2 x \rfloor$ and $\tilde{y} = \lfloor k \log_2 y \rfloor$, where k is an integer scaling factor that controls quantization granularity. Then:

$$x \cdot y \approx 2^{(\tilde{x}+\tilde{y})/k}, \qquad \frac{x}{y} \approx 2^{(\tilde{x}-\tilde{y})/k} \quad (4)$$

If projected indices are quantized to 256 levels (8 bits per operand), the resulting lookup table contains only $256 \times 256 = 65{,}536$ entries (64 KB)—easily fitting within on-chip SRAM.

4.3.2 Lookup-Based Computation Pipeline

BurstMon implements logarithmic projection in a three-stage pipeline (Figure 6): **1. Log Table Lookup:** Retrieve $\lfloor k \log_2 x \rfloor$

and $\lfloor k \log_2 y \rfloor$ from precomputed tables. **2. Integer Arithmetic:** Perform addition $(\tilde{x}+\tilde{y})$ or subtraction $(\tilde{x}-\tilde{y})$ to compute the projected exponent. **3. Exponentiation Lookup:** Index into an exponentiation table to retrieve $2^{(\tilde{x}\pm\tilde{y})/k}$.

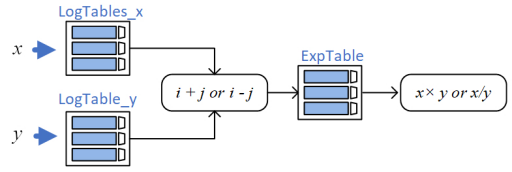


Figure 6: Approximate computation of multiplication and division via logarithmic projection and table lookup.

Using this mechanism, BurstMon approximates the full burst score $|a_i \cdot i - s_i| / \sqrt{s_i(i-1)}$ with **9 lookup operations** across **7 pipeline stages**, as shown in Figure 7. The inverse square root term is decomposed into a precomputed **Inverse Square Root (ISR)** component for improved accuracy.

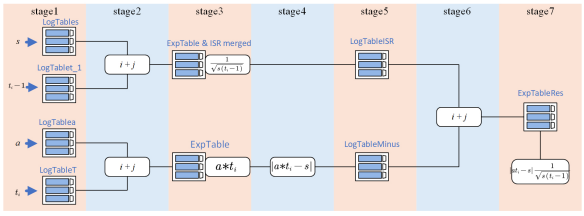


Figure 7: Pipeline design for in-data-plane burst score approximation.

4.3.3 Burst Detection Decision

Once the burst score is computed, BurstMon compares it against a configurable threshold T. If score $\geq T$, a burst is detected and the data plane emits a `digest` containing the flow ID, timestamp, and original timeslot volume values (r_i, r_{i-1}). If score $< T$, no report is generated and the flow continues to be monitored silently.

4.3.4 False positive bound.

We derive a probabilistic upper bound on the false positive rate by combining two sources of error: the statistical variability of the chi-square test and the estimation error of the Count-Min Sketch.

Proposition 1 (False Positive Bound)**.** *Let $\chi^2_{1-\alpha}$ denote the $(1-\alpha)$ quantile of the chi-square distribution with 1 degree of freedom. Configure the CMS with width w and depth d such that the per-query failure probability $\delta_{\text{cms}} = e^{-d}$ satisfies $\delta_{\text{cms}} \leq \alpha/2$. Then under the null hypothesis H_0 (no burst, i.e., a_i is consistent with the historical mean), the estimated burst statistic $\hat{\chi}^2$ satisfies:*

$$P\left(\hat{\chi}^2 > \chi^2_{1-\alpha/2}\right) < \alpha$$

Proof in Appendix A.1

4.4 Control-Plane Reconstruction (Q4)

When a burst is detected, BurstMon transmits minimal metadata to the control plane via `digest` or `truncated mirror`. Each report includes the flow ID, timestamp, and rate values from two consecutive timeslots. The control plane uses this information for rate curve reconstruction and behavioral analysis.

Interpolated rate curve reconstruction. Given two burst reports at timeslots t_1 and t_2 with observed rates r_1 and r_2, the interpolated flow rate at any intermediate time $t \in [t_1, t_2]$ is:

$$r(t) = r_1 + \frac{r_2 - r_1}{t_2 - t_1} \cdot (t - t_1) \tag{5}$$

This method achieves over 95% cosine similarity with ground-truth traces (Section 5.2). The justification for linear interpolation is that BurstMon's χ^2 test suppresses reports precisely during periods where the flow rate is stable (Appendix A.2), so a linear assumption between consecutive change points closely matches ground truth.

Handling sustained bursts and extended stable periods. A natural question is how BurstMon handles a burst that persists across multiple timeslots—for example, a flow that jumps from 10 Gbps to 80 Gbps and remains there before dropping back. BurstMon detects the *onset* at the first timeslot where the rate changes sharply (large a_i, producing a high burst score), and the *offset* when the rate drops back. During the elevated-rate plateau, the per-timeslot rate differences a_i are near zero, so no additional reports are generated. The control plane receives the onset and offset reports and linearly interpolates between them—producing a flat line at the elevated rate, which faithfully reflects the true behavior.

Conversely, if a flow remains at a steady rate for an extended period with no burst reports, the control plane holds the last reported rate as the current estimate (*constant extrapolation*). This is sound because the absence of a report certifies that every intervening timeslot satisfied score$(f,i) \leq T$, which by Equation 11 constrains the true rate to within $[r_{\text{last}} - a_i^{\max},\ r_{\text{last}} + a_i^{\max}]$. The reconstruction error during any unreported period is therefore formally bounded.

Microsecond-resolution behavioral analysis. BurstMon's temporal resolution enables fine-grained network analyses including congestion attribution, pulse-wave DDoS detection, and real-time elephant flow steering, demonstrated in Section 5.4.

5 Experimental Evaluation

We implement BurstMon on a Barefoot Tofino1 programmable switch and evaluate it using production-inspired workloads. Our experiments address five questions:

RQ1: How accurately does the lookup-table-based logarithmic projection replicate true floating-point burst score computations, and how does the scaling factor k affect accuracy and memory? (§5.1)
RQ2: How well can BurstMon reconstruct microsecond-granularity per-flow rate curves compared to existing systems, and what are the resulting communication overhead and response latency? (§5.2)
RQ3: How does the burst score threshold T affect reconstruction accuracy, uplink bandwidth, and robustness across diverse workloads? (§5.3)
RQ4: Can BurstMon's microsecond-resolution telemetry improve real-world network tasks, including congestion attribution, attack detection, traffic engineering, and early congestion signaling? (§5.4)
RQ5: What are the on-chip resource requirements and forwarding throughput impact of deploying BurstMon on commodity hardware? (§5.5)

5.1 Lookup-Table Approximation Accuracy (RQ1)

Setup. We deploy BurstMon on a Tofino1 testbed and compare its in-switch burst score computations against CPU-based ground truth using standard IEEE 754 [23] 16-bit floating-point arithmetic. To our knowledge, BurstMon is the first system to support a complete chi-square statistical test—including division and square root—entirely within a programmable switch pipeline.

Metrics. We adopt Mean Squared Error (MSE) and an exponential-form Accuracy score over $N = 10{,}000$ consecutive timeslot samples:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(S_i^{\text{CPU}} - S_i^{\text{DP}}\right)^2, \quad \text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\exp\left(-\frac{|\Delta_i|}{S_i^{\text{CPU}}}\right) \tag{6}$$

where S_i^{CPU} and S_i^{DP} are the burst scores computed on the CPU and in the data plane, respectively, and $\Delta_i = S_i^{\text{CPU}} - S_i^{\text{DP}}$.

Results. The scaling factor k directly impacts both approximation accuracy and memory overhead. Figure 8 shows that as k increases, MSE decreases and Accuracy increases: at $k = 2048$, MSE falls to 0.0001 and Accuracy reaches 0.9969, but the table size exceeds 5×10^5 entries (636 KB). To balance accuracy and resource usage, we choose $k = 512$ for all subsequent experiments, achieving MSE < 0.01 and Accuracy > 0.995 with a practical memory footprint. Once set, k remains fixed across all experiments and deployments.

5.2 Flow Rate Reconstruction (RQ2)

Testbed. We deploy BurstMon on a testbed consisting of a Tofino1 switch and two 100 Gbps NICs connected via high-speed links. Packet-level traffic is generated using NS-3 simulations and replayed through the switch at line rate. BurstMon

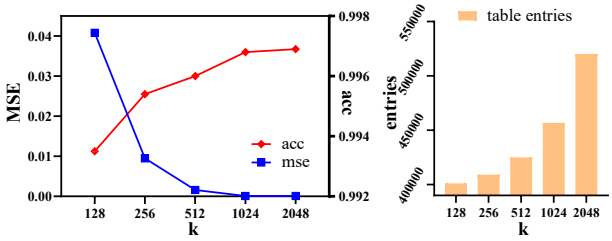


Figure 8: Impact of scaling factor k on approximation error and table size.

detects bursts in the data plane and reports only burst events to the control plane, which reconstructs flow rate curves via linear interpolation. We retain the ground-truth flow rate curves for accuracy evaluation.

Traffic workloads. We use four workloads with diverse traffic characteristics: (i) `Hadoop15`, simulated at 15% average link utilization with 24 ms trace duration; (ii) `WebSearch25`, at 25% utilization with 31 ms duration; (iii) `IXPTrace`, a real-world Internet Exchange Point trace with 12 ms duration; and (iv) `MSTrace`, a 20 ms trace simulated in NS-3 with flow CDF derived from Microsoft data center distributions [2]. To evaluate robustness under high load, we set traffic utilization to 80%. Table 2 summarizes the per-flow statistics. We quantify

Table 2: Flow statistics for the four workloads.

Trace	Timestep (μs)	Duration (ms)	Crest Factor
Hadoop15	10	24.76	2.20
WebSearch25	10	31.64	1.13
IXPTrace	80	12.98	2.62
MSTrace	80	19.86	2.10

volatility using the Crest Factor (CF), defined as the ratio of a flow's peak rate to its root-mean-square rate. Three of four traces exhibit CF exceeding 2.0, with IXPTrace reaching 2.62, indicating high volatility. In contrast, WebSearch25 has a CF of 1.13, representing steady-state behavior. This diversity validates BurstMon's applicability across different network environments.

Figure 10 presents the CDF of absolute first-order rate differences across all four workloads. In every case, the CDF rises sharply near zero, confirming the sparsity of burst events and validating BurstMon's core design assumption of event-domain compressibility.

Metrics and settings. The timeslot is fixed at 10 μs for Hadoop15 and WebSearch25 workloads and at 80 μs for IXPTrace and MSTrace. All systems are evaluated under equivalent memory budgets (e.g., 256 KB). We measure reconstruction accuracy of each flow via Cosine Similarity (COS), Euclidean Distance (ED), Energy ratio, and Relative Peak

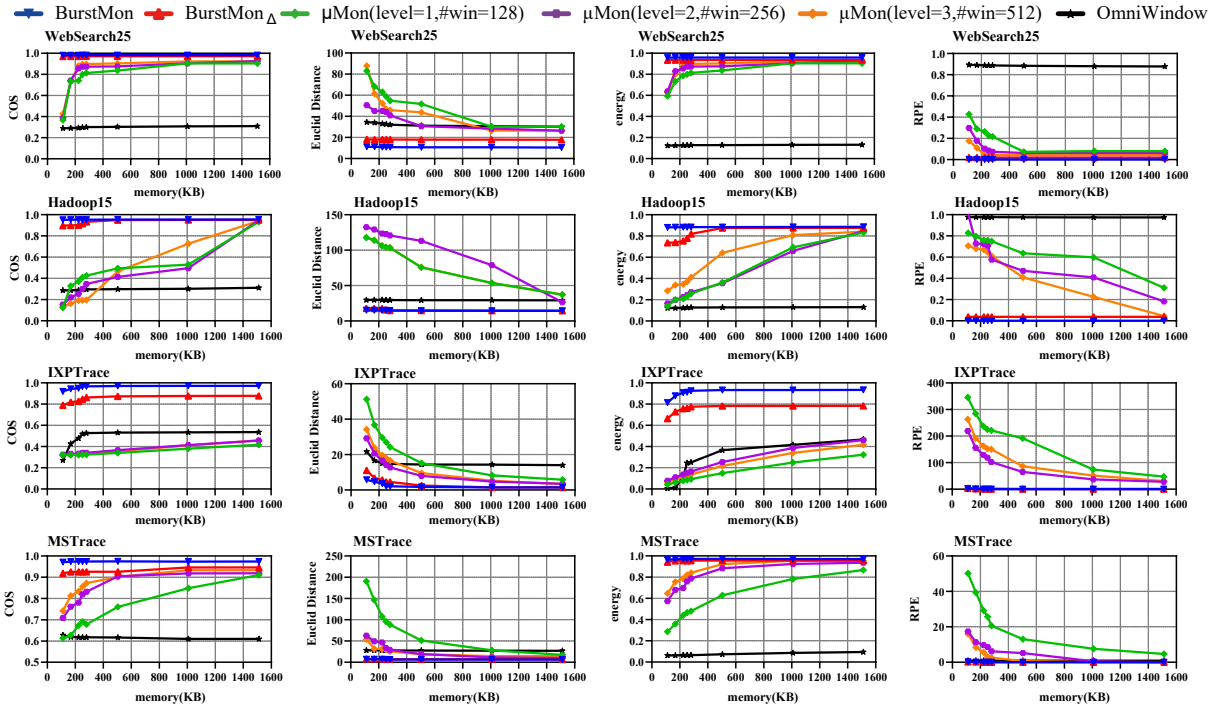


Figure 9: Flow rate reconstruction accuracy under four workloads. BurstMon consistently outperforms baselines across memory budgets on all metrics.

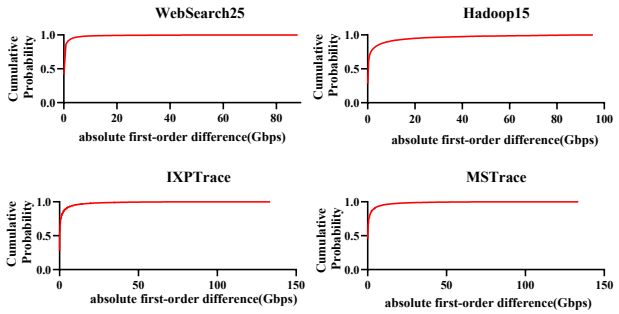


Figure 10: CDF of absolute first-order rate differences across four workloads, confirming burst sparsity.

Estimation Error (RPE):

$$ED = \sqrt{\sum_{t=1}^{N}(f_t - \hat{f}_t)^2}, \quad COS = \frac{\sum_{t=1}^{N} f_t \cdot \hat{f}_t}{\sqrt{\sum_{t=1}^{N} f_t^2}\sqrt{\sum_{t=1}^{N} \hat{f}_t^2}},$$
$$Energy = \min\left(\frac{\sum f_t^2}{\sum \hat{f}_t^2}, \frac{\sum \hat{f}_t^2}{\sum f_t^2}\right), \quad RPE = \frac{|\max(f) - \max(\hat{f})|}{\max(\hat{f})} \tag{7}$$

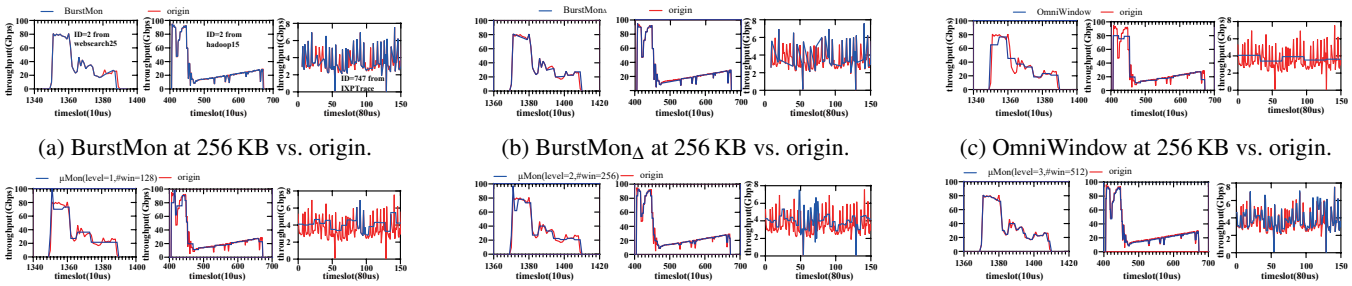
where f_t and $\hat{f}_t$ denote the original and reconstructed flow rates at timeslot t for flow f. We then report the mean metric of flows in each workload.

Baselines. We compare against three systems: **μMon** [33], which compresses flow statistics using Haar wavelets across multiple timeslots (high accuracy but buffering latency); **OmniWindow** [26], which reports averaged values over multiple timeslots to limit uplink usage (reduced accuracy); and **BurstMon$_\Delta$**, a simplified variant that detects bursts via a fixed first-order rate threshold ($a_i > T_\Delta$) without chi-square normalization.

Reconstruction accuracy. As shown in Figure 9, BurstMon consistently outperforms all baselines across memory budgets. On `Hadoop15,T=200`, it achieves COS > 0.97, ED $\sim$10, Energy ≈ 1.0, and RPE < 0.001. On `WebSearch25,T=256`, COS > 0.95, ED < 20, Energy > 0.90, and RPE < 0.001. On the more challenging `IXPTrace,T=128` (CF > 2.5), BurstMon achieves COS > 0.92 and notably outperforms BurstMon$_\Delta$ by $\sim$0.1 in COS, demonstrating the value of chi-square normalization for highly volatile traffic. On `MSTrace,T=128`, COS > 0.97, ED < 7, and Energy > 0.96.

While μMon benefits from deep wavelet hierarchies (128–512 timeslots), it suffers from high latency. OmniWindow's temporal smoothing reduces fidelity. BurstMon$_\Delta$ outperforms both but still lags behind full BurstMon due to its fixed threshold being insensitive to per-flow variability. Figure 11 visually confirms that BurstMon closely tracks ground-truth flow dynamics.

Communication overhead. Figure 12a shows that BurstMon maintains low and stable communication overhead: 0.05 Gbps on `Hadoop15` and 0.07 Gbps on `WebSearch25`. This overhead is independent of sketch memory size and scales only with burst frequency for a given threshold T—a key advantage of event-driven reporting over periodic sketch



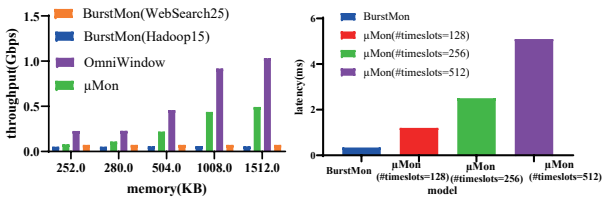
(a) BurstMon at 256 KB vs. origin. (b) BurstMon$_\Delta$ at 256 KB vs. origin. (c) OmniWindow at 256 KB vs. origin.

(d) μMon (level=1, 128 timeslots) vs. origin. (e) μMon (level=2, 256 timeslots) vs. origin. (f) μMon (level=3, 512 timeslots) vs. origin.

Figure 11: Reconstructed vs. original flow rate curves under different systems.

uploads. In contrast, μMon and OmniWindow adopt periodic sketch-upload models that incur significantly higher and less predictable overhead.

Response latency. We define response latency as the end-to-end time from the timeslot in which a burst occurs to the availability of the reported flow data on the control plane. As shown in Figure 12b, μMon incurs buffering delays of 1.2 ms (128 timeslots), 2.5 ms (256), and 5.1 ms (512) due to wavelet compression requiring multiple timeslots of accumulated data. In contrast, BurstMon reports bursts within the same timeslot they are detected. With the timeslot set to 10 μs, BurstMon achieves a response latency of 410 μs. Its end-to-end latency is dominated by the timeslot aggregation interval ($\sim$10 μs), switch pipeline processing ($\sim$340 ns), and digest uplink transfer ($\sim$100 μs),and the flow reconstruction latency after the control plane receives the digest information($\sim$300μs), totaling approximately 410μs—much faster than μMon's most aggressive configuration. This low latency enables timely fine-grained traffic analytics that are infeasible with wavelet-based approaches.



(a) Uplink comm. overhead. (b) Response latency.

Figure 12: Response latency and uplink communication overhead comparison.

Reconstruction of highly fluctuating small flows. For small flows characterized by frequent fluctuations, the upper bound of the tolerance region (Equation 12) approximates $\bar{a}+T\sqrt{\bar{a}}$, which is independent of flow size. BurstMon therefore remains capable of detecting bursts within these flows. Figure 13 illustrates the reconstruction fidelity for such flows from the IXPTrace, confirming that BurstMon handles volatile small flows well.

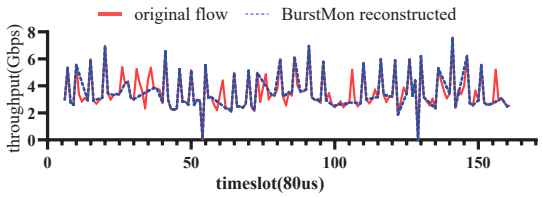
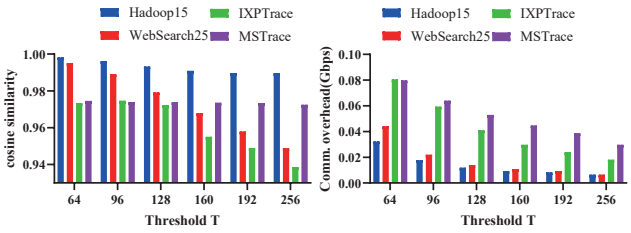


Figure 13: BurstMon reconstruction for a frequently fluctuating small flow in IXPTrace(80μs,1512KB,T=128) .

5.3 Impact of Burst Score Threshold T (RQ3)

The threshold T directly controls the trade-off between detection sensitivity and communication cost. A smaller T captures finer-grained variations but increases reporting frequency; a larger T suppresses minor bursts but may miss meaningful dynamics. We evaluate this trade-off on all four workloads with sketch memory fixed at 1512 KB.



(a) COS similarity under varying threshold T. (b) Comm. overhead under varying threshold T.

Figure 14: COS similarity and comm. overhead under varying threshold T.

Figure 14 evaluates BurstMon across four workloads, demonstrating its robustness and efficiency. As shown in Figure 14a, even at the maximum threshold $T = 256$, the COS similarity remains above 0.94 for all workloads. This consistency highlights BurstMon's robustness against diverse traffic burstiness (Crest Factors ranging from 1.13 to 2.62). The self-normalizing property of the χ^2-based burst score ensures that a single global T automatically provides tighter sensitivity for stable flows and wider tolerance for volatile ones, making

reliable detection possible across diverse network dynamics without per-workload parameter tuning. In terms of overhead (Figure 14b), increasing T significantly reduces uplink bandwidth. Even at $T = 64$, the maximum overhead remains below 0.08 Gbps, well below the 10 Gbps digest interface capacity, confirming deployability in high-speed production environments.

Threshold configuration. In practice, operators can configure T analytically based on network anomaly targets.

As the flow progresses ($i \to \infty$) and s_i grows proportionally, the average fluctuation converges to $\bar{a} = s_i/i$. The tolerance region bound simplifies to

$$a_i^{\max} \sim \bar{a} + T \cdot \sqrt{\bar{a}} \tag{8}$$

meaning that flow rate r_i in $[r_{i-1} - a_i^{\max},\ r_{i-1} + a_i^{\max}]$ will not be detected as bursts.Detailed analysis is in Appendix A.2.

Given a link with typical background fluctuation $\bar{a}_{bg}$, to capture any microburst reaching a critical deviation Δa_{target} (e.g., triggering ECN marking or PFC pauses), T can be set as:

$$T = \frac{\Delta a_{\text{target}} - \bar{a}_{bg}}{\sqrt{\bar{a}_{bg}}} \tag{9}$$

This formula follows directly from inverting Equation 8: the threshold T is chosen so that the tolerance zone boundary $a_i^{\max}$ coincides with the target deviation Δa_{target}. We empirically set $T = 200$ for `Hadoop15`, $T = 256$ for `WebSearch25`, and $T = 128$ for `IXPTrace` and `MSTrace`. At deployment time, a brief 1 ms traffic sample suffices to estimate $\bar{a}_{bg}$ and calibrate T for each workload.

Robustness to workload drift. A practical concern is whether a fixed threshold T remains effective as traffic characteristics evolve over time. We note two complementary sources of robustness. First, the χ^2-based burst score is *self-normalizing*: the denominator $\sqrt{s_i(i-1)}$ adapts to each flow's cumulative variability, so a single global T automatically provides tighter sensitivity for stable flows and wider tolerance for volatile ones (Equation 12). This per-flow adaptivity is inherent and requires no parameter change. Second, if the aggregate traffic mix shifts substantially (e.g., due to a new application deployment), the control plane can periodically re-estimate $\bar{a}_{bg}$ from recent burst report statistics—for instance, by computing the mean reported a_i over the last 1–10 ms—and update T via Equation 9. Because T is a single 16-bit register value read by the data plane at each comparison, updating it incurs zero pipeline disruption and takes effect immediately. In our experiments, we observe that a fixed T already performs well across all four workloads (COS > 0.92 in every case), suggesting that frequent recalibration is unnecessary for typical data-center traffic.

5.4 Downstream Application Use Cases (RQ4)

A key claim of BurstMon is that microsecond-resolution rate curve reconstruction enables network analytics that are infeasible at coarser granularity. We validate this claim through four application scenarios spanning diagnostics, security, traffic engineering, and congestion management.

Table 3: Culprit flow identification performance at 80 μs granularity.

Method	M-Prec	M-Rec	M-F1	W-Prec	W-Rec	W-F1	Overlap
Ground Truth	1.000	1.000	1.000	1.000	1.000	1.000	10.0
BurstMon	**0.938**	**0.938**	**0.938**	**0.910**	**0.910**	**0.910**	**9.38**
μMon	0.619	0.619	0.619	0.615	0.615	0.615	6.19
Sampling (1 ms)	0.500	0.500	0.500	0.471	0.471	0.471	5.00

Use Case 1: Fine-grained congestion attribution. BurstMon's microsecond-granularity traffic reconstruction, combined with ECN marking logs, enables precise attribution of congestion-contributing flows. Figure 15 illustrates a representative congestion episode: before onset ($t < 0$), the red flow maintains steady throughput at 60–100 Gbps. At $t = 0$, the green flow injects a sharp burst ramping to ∼130 Gbps within hundreds of microseconds, triggering queue buildup and ECN marking. A second burst from the blue flow at $t \approx 1$ ms prolongs congestion to ∼3 ms total duration. ECN feedback drives sequential rate reduction across all flows, restoring normal utilization.

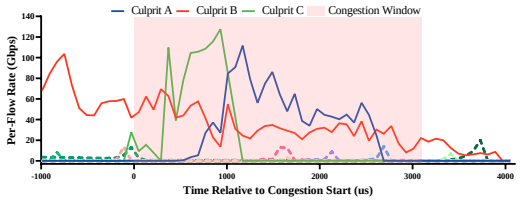


Figure 15: Fine-grained congestion analysis using BurstMon.

Table 3 compares top-K culprit flow identification across methods. BurstMon achieves macro-F1 of 0.938 with 9.38 out of 10 culprit flows correctly identified, closely approaching ground truth and outperforming μMon (0.619) and 1 ms sampling (0.500). The advantage stems from BurstMon's ability to resolve the *temporal ordering* of burst onsets—information that is smoothed away at coarser granularity.

Use Case 2: Pulse-wave DDoS detection. We evaluate BurstMon against a 1 ms sliding window baseline on synthetic pulse-wave DDoS traffic—periodic bursts lasting tens of microseconds followed by silent intervals. Figure 16 shows that BurstMon's 80 μs granularity faithfully reconstructs the attack profile, exposing peak rates of ∼11 Gbps during burst phases. The 1 ms baseline averages these spikes to only ∼1 Gbps—indistinguishable from benign traffic. This demonstrates that sub-millisecond visibility is essential for detecting modern evasive attacks that exploit temporal averaging.

Use Case 3: Elephant flow steering. We demonstrate BurstMon's online elephant flow detection and steering capability with a simple closed-loop scenario. BurstMon monitors per-flow rate changes in the data plane and, upon detecting

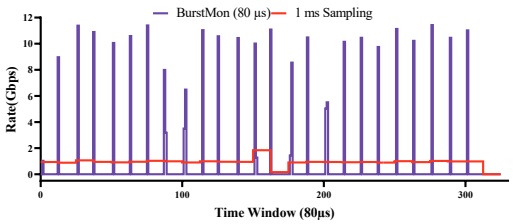


Figure 16: Pulse-wave DDoS detection: BurstMon (80 μs) vs. 1 ms sampling.

Table 4: Baseline (ECMP) vs. BurstMon-assisted steering.

Metric	ECMP	BurstMon	Improvement
Total TCP Flows	1126	1126	–
Completed Flows	1125	1126	+1 flow
Mean FCT (s)	1.7580	1.6325	7.14%
P99 FCT (s)	11.5561	10.1639	12.05%
Lost Packets	24,277	22,237	8.40%
Packet Drop Rate	0.218%	0.200%	8.26%

an elephant flow, reports its identifier and rate to the control plane via `digest`; the control plane then reassigns the flow to a less-loaded path. We evaluate in NS-3 with a leaf-spine topology (3 spine, 4 leaf switches, 8 hosts per leaf), Poisson background traffic under ECMP, and sub-millisecond microburst injection.

As shown in Table 4, even with this simple strategy, BurstMon-assisted steering reduces P99 tail latency by 12.05%, mean FCT by 7.14%, and packet drop rate from 0.218% to 0.200%, with all TCP flows completing successfully. These results confirm that BurstMon's microsecond-level rate awareness effectively supports online elephant flow steering.

Use Case 4: Proactive congestion signaling. Detailed in Appendix B due to space constraints.

Summary. Across four diverse application scenarios, BurstMon's microsecond-resolution rate curves enable analyses that are fundamentally infeasible at millisecond granularity: resolving the causal ordering of congestion contributors, exposing attack peaks hidden by temporal averaging, providing timely feedback for closed-loop traffic engineering, and detecting congestion proactively rather than reactively.

5.5 Switch Resource & Throughput (RQ5)

Hardware resource consumption. Table 5 summarizes BurstMon's resource footprint on Tofino1. Resource utilization remains low across all categories: hash-bit usage is 13.18% and all other computational components are below 10%; SRAM is the most utilized memory at 26.04% with no TCAM consumed; PHV usage is 11.65%. This leaves substantial headroom for future scaling in the data plane.

Forwarding throughput. We measure port throughput under two configurations: with BurstMon enabled and with basic L2 forwarding only. Using `pktgen`, we generate high-speed traffic for 30 seconds and record receiver throughput.

Table 5: Resource consumption of BurstMon on Tofino1.

Computational	eMatch xBar	tMatch xBar	Gateway	VLIW	Hash bits
Usage (%)	4.10	0.00	6.77	6.25	13.18
Memory	SRAM	TCAM	Map RAM	Table ID	Stash
Usage (%)	26.04	0.00	3.47	19.27	5.73
PHV	8-bit	16-bit	32-bit	Overall	
Usage (%)	6.64	22.01	4.69	11.65	
PHV (T)	8-bit	16-bit	32-bit	Overall	
Usage (%)	9.38	12.50	12.50	11.65	

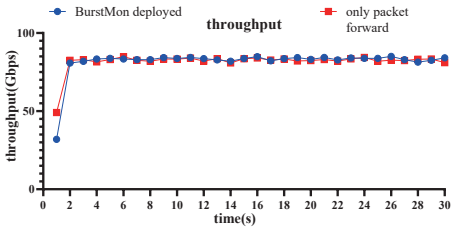


Figure 17: Port throughput with/without BurstMon enabled.

As shown in Figure 17, throughput remains stable at ~80 Gbps in both configurations. The small dip in the first second (~50 Gbps) is due to NIC cold-start, not switch logic. BurstMon introduces negligible forwarding overhead despite performing microsecond-scale monitoring and burst detection entirely in the data plane.

6 Conclusion

We presented BurstMon, a microsecond-level monitoring system that combines data-plane burst detection with control-plane interpolation to deliver fine-grained, per-flow visibility at line rate. The design is grounded in the observation that microsecond-scale flow rate signals are sparse in the *event domain*—long stable periods punctuated by a small number of change points—enabling a telemetry architecture that reports events rather than compressing measurements.

BurstMon introduces three key techniques: a χ^2-based burst detector implemented with lookup-table arithmetic, a rotating time-sketch with dual-purpose timestamps for per-flow measurement, and a sparse-report reconstruction pipeline. Our Tofino-1 prototype achieves over 95% cosine similarity across four diverse workloads while holding control-plane traffic below 0.07 Gbps and imposing negligible forwarding overhead. Four practical use cases—congestion attribution, pulse-wave DDoS detection, elephant flow steering, and proactive congestion signaling—demonstrate the value of microsecond-resolution temporal context for network analytics.

A natural direction for future work is closing the loop on threshold selection: because the burst score threshold T is a single register value that can be updated at runtime with zero pipeline disruption, a control-plane feedback controller could continuously adjust T so that the monitoring bandwidth converges to a specified budget. This would free operators from manual tuning and automatically maximize telemetry fidelity under any given uplink capacity constraint.

References

[1] Inayat Ali, Seungwoo Hong, Pyungkoo Park, and Tae Yeon Kim. Rethinking explicit congestion notification: A multilevel congestion feedback perspective. In *Proceedings of the 34th edition of the Workshop on Network and Operating System Support for Digital Audio and Video, NOSSDAV 2024, Bari, Italy, April 15-18, 2024*, pages 64–70. ACM, 2024.

[2] Mohammad Alizadeh, Albert G. Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. Data center TCP (DCTCP). In *Proceedings of the ACM SIGCOMM 2010 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, New Delhi, India, August 30 -September 3, 2010*, pages 63–74. ACM, 2010.

[3] Ran Ben Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minlan Yu, and Michael Mitzenmacher. PINT: probabilistic in-band network telemetry. In *SIGCOMM '20: Proceedings of the 2020 Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication, Virtual Event, USA, August 10-14, 2020*, pages 662–680. ACM, 2020.

[4] Daniel Baumgartner and John E. Kolassa. Power considerations for Kolmogorov-Smirnov and Anderson-Darling two-sample tests. *Commun. Stat. Simul. Comput.*, 52(7):3137–3145, 2023.

[5] Siddharth Bhatia, Bryan Hooi, Minji Yoon, Kijung Shin, and Christos Faloutsos. Midas: Microcluster-based detector of anomalies in edge streams. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 3242–3249, 2020.

[6] Xiaoqi Chen, Shir Landau Feibish, Yaron Koral, Jennifer Rexford, and Ori Rottenstreich. Catching the Microburst Culprits with Snappy. In *Proceedings of the Afternoon Workshop on Self-Driving Networks, SelfDN@SIGCOMM 2018, Budapest, Hungary, August 24, 2018*, pages 22–28. ACM, 2018.

[7] Xiaoqi Chen, Shir Landau Feibish, Yaron Koral, Jennifer Rexford, Ori Rottenstreich, Steven A. Monetti, and Tzuu-Yi Wang. Fine-grained queue measurement in the data plane. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies, CoNEXT 2019, Orlando, FL, USA, December 09-12, 2019*, pages 15–29. ACM, 2019.

[8] Benoit Claise. Cisco systems NetFlow services export version 9. *RFC*, 3954:1–33, 2004.

[9] Zicun Cong, Lingyang Chu, Yu Yang, and Jian Pei. Comprehensible Counterfactual Explanation on Kolmogorov-Smirnov Test. *Proc. VLDB Endow.*, 14(9):1583–1596, 2021.

[10] Penglai Cui, Heng Pan, Zhenyu Li, Jiaoren Wu, Shengzhuo Zhang, Xingwu Yang, Hongtao Guan, and Gaogang Xie. NetFC: Enabling accurate floating-point arithmetic on programmable switches. In *2021 IEEE 29th International Conference on Network Protocols (ICNP)*, pages 1–11. IEEE, 2021.

[11] Rui Ding, Shibo Yang, Xiang Chen, and Qun Huang. BitSense: Universal and Nearly Zero-Error Optimization for Sketch Counters with Compressive Sensing. In *Proceedings of the ACM SIGCOMM 2023 Conference, ACM SIGCOMM 2023, New York, NY, USA, 10-14 September 2023*, pages 220–238. ACM, 2023.

[12] Shir Landau Feibish, Zaoxing Liu, Nikita Ivkin, Xiaoqi Chen, Vladimir Braverman, and Jennifer Rexford. Flow-level loss detection with Δ-sketches. In *SOSR '22: The ACM SIGCOMM Symposium on SDN Research, Virtual Event, October 19 - 20, 2022*, pages 25–32. ACM, 2022.

[13] Marco Gaboardi and Ryan Rogers. Local private hypothesis testing: Chi-square tests. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 1612–1621. PMLR, 2018.

[14] Ehab Ghabashneh, Yimeng Zhao, Cristian Lumezanu, Neil Spring, Srikanth Sundaresan, and Sanjay G. Rao. A microscopic view of bursts, buffer contention, and loss in data centers. In *Proceedings of the 22nd ACM Internet Measurement Conference, IMC 2022, Nice, France, October 25-27, 2022*, pages 567–580. ACM, 2022.

[15] Qun Huang, Siyuan Sheng, Xiang Chen, Yungang Bao, Rui Zhang, Yanwei Xu, and Gong Zhang. Toward nearly-zero-error sketching via compressive sensing. In *18th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2021, April 12-14, 2021*, pages 1027–1044. USENIX Association, 2021.

[16] Raj Joshi, Ting Qu, Mun Choon Chan, Ben Leong, and Boon Thau Loo. BurstRadar: Practical real-time microburst monitoring for datacenter networks. In *Proceedings of the 9th Asia-Pacific Workshop on Systems*, pages 1–8, 2018.

[17] Ike Kunze, Moritz Gunz, David Saam, Klaus Wehrle, and Jan Rüth. Tofino + P4: A strong compound for AQM on high-speed networks? In *17th IFIP/IEEE International Symposium on Integrated Network Management, IM 2021, Bordeaux, France, May 17-21, 2021*, pages 72–80. IEEE, 2021.

[18] Aleksandar Kuzmanovic. The power of explicit congestion notification. In *Proceedings of the ACM SIGCOMM 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, Philadelphia, Pennsylvania, USA, August 22-26, 2005*, pages 61–72. ACM, 2005.

[19] Yiran Lei, Liangcheng Yu, Vincent Liu, and Mingwei Xu. PrintQueue: Performance diagnosis via queue measurement in the data plane. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 516–529, Amsterdam Netherlands, August 2022. ACM.

[20] Linxi Li, Kun Xie, Shuyu Pei, Jigang Wen, Wei Liang, and Gaogang Xie. CS-Sketch: Compressive Sensing Enhanced Sketch for Full Traffic Measurement. *IEEE Trans. Netw. Sci. Eng.*, 11(3):2338–2352, 2024.

[21] Kefei Liu, Zhuo Jiang, Jiao Zhang, Shixian Guo, Xuan Zhang, Yangyang Bai, Yongbin Dong, Feng Luo, Zhang Zhang, Lei Wang, Xiang Shi, Haohan Xu, Yang Bai, Dongyang Song, Haoran Wei, Bo Li, Yongchen Pan, Tian Pan, and Tao Huang. R-Pingmesh: A Service-Aware RoCE Network Monitoring and Diagnostic System. In *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024*, pages 554–567. ACM, 2024.

[22] Zaoxing Liu, Antonis Manousis, Gregory Vorsanger, Vyas Sekar, and Vladimir Braverman. One Sketch to Rule Them All: Rethinking Network Flow Monitoring with UnivMon. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 101–114, 2016.

[23] Peter Markstein. The new IEEE-754 standard for floating point arithmetic. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2008.

[24] Michael P. Oakes, Robert J. Gaizauskas, and Helene Fowkes. A method based on the chi-square test for document classification. In *SIGIR 2001: Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, September 9-13, 2001, New Orleans, Louisiana, USA*, pages 440–441. ACM, 2001.

[25] Peter Phaal, Sonia Panchen, and Neil McKee. InMon Corporation's sFlow: A method for Monitoring Traffic in Switched and Routed Networks. *RFC*, 3176:1–31, 2001.

[26] Haifeng Sun, Jiaheng Li, Jintao He, Jie Gui, and Qun Huang. OmniWindow: A General and Efficient Window Mechanism Framework for Network Telemetry. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 867–880, 2023.

[27] Weitao Wang, Xinyu Crystal Wu, Praveen Tammana, Ang Chen, and T. S. Eugene Ng. Closed-loop network performance monitoring and diagnosis with spidermon. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*, pages 267–285. USENIX Association, 2022.

[28] Jinzhu Yan, Haotian Xu, Zhuotao Liu, Qi Li, Ke Xu, Mingwei Xu, and Jianping Wu. Brain-on-switch: Towards advanced intelligent network data plane via nn-driven traffic analysis at line-speed. In *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, pages 419–440. USENIX Association, 2024.

[29] Siyu Yan, Xiaoliang Wang, Xiaolong Zheng, Yinben Xia, Derui Liu, and Weishan Deng. ACC: Automatic ECN tuning for high-speed datacenter networks. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 384–397, Virtual Event USA, August 2021. ACM.

[30] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. Elastic sketch: Adaptive and fast network-wide measurements. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 561–575, 2018.

[31] Yinda Zhang, Zaoxing Liu, Ruixin Wang, Tong Yang, Jizhou Li, Ruijie Miao, Peng Liu, Ruwen Zhang, and Junchen Jiang. CocoSketch: High-performance sketch-based measurement over arbitrary partial key query. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 207–222, 2021.

[32] Yikai Zhao, Kaicheng Yang, Zirui Liu, Tong Yang, Li Chen, Shiyi Liu, Naiqian Zheng, Ruixin Wang, Hanbo Wu, Yi Wang, et al. LightGuardian: A full-visibility, lightweight, in-band telemetry system using sketchlets. In *18th USENIX symposium on networked systems design and implementation (NSDI 21)*, pages 991–1010, 2021.

[33] Hao Zheng, Chengyuan Huang, Xiangyu Han, Jiaqi Zheng, Xiaoliang Wang, Chen Tian, Wanchun Dou, and Guihai Chen. μMon: Empowering Microsecond-level Network Monitoring with Wavelets. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 274–290, Sydney NSW Australia, August 2024. ACM.

[34] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. An efficient design of intelligent network data plane. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, pages 6203–6220. USENIX Association, 2023.

[35] Huancheng Zhou and Guofei Gu. Cerberus: Enabling efficient and effective in-network monitoring on programmable switches. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*, pages 4424–4439. IEEE, 2024.

A Analysis and Theoretical Guarantees

We analyze BurstMon's detection reliability, reconstruction fidelity, and approximation accuracy through two formal results.

A.1 False Positive Bound

We derive a probabilistic upper bound on the false positive rate by combining two sources of error: the statistical variability of the chi-square test and the estimation error of the Count-Min Sketch.

Setup and notation. Let r_i and $\hat{r}_i$ denote the true and CMS-estimated flow rates at timeslot i; $a_i = |r_i - r_{i-1}|$ and $\hat{a}_i = |\hat{r}_i - \hat{r}_{i-1}|$ the true and estimated first-order differences; $s_i = \sum_{k=1}^{i} |r_k - r_{k-1}|$ and $\hat{s}_i$ the true and CMS-estimated cumulative variations; and n the number of timeslots observed so far. The CMS with width w and depth d satisfies the standard error model: for any individual query, $\hat{r}_i \le r_i + \frac{e}{w} N_i$ with probability at least $1 - e^{-d}$, where N_i is the total traffic volume in timeslot i and e is the base of the natural logarithm.

Proposition 1 (False Positive Bound)**.** *Let $\chi^2_{1-\alpha}$ denote the $(1-\alpha)$ quantile of the chi-square distribution with 1 degree of freedom. Configure the CMS with width w and depth d such that the per-query failure probability $\delta_{\text{cms}} = e^{-d}$ satisfies $\delta_{\text{cms}} \le \alpha/2$. Then under the null hypothesis H_0 (no burst, i.e., a_i is consistent with the historical mean), the estimated burst statistic $\hat{\chi}^2$ satisfies:*

$$P\left(\hat{\chi}^2 > \chi^2_{1-\alpha/2}\right) < \alpha$$

Proof. The proof proceeds by bounding the two error sources separately and combining them via a union bound.

Step 1: CMS error conditioning. Define the *good event* $\mathcal{G}$: all CMS queries involved in computing $\hat{a}_i$ and $\hat{s}_i$ return values within the standard overestimation guarantee. By a union bound over the $O(1)$ queries per timeslot, $P(\mathcal{G}) \ge 1 - \delta_{\text{cms}} \ge 1 - \alpha/2$. Conditioned on $\mathcal{G}$, each CMS estimate satisfies $0 \le \hat{r}_k - r_k \le \frac{e}{w} N_k$. By the triangle inequality applied to $\hat{a}_i = |\hat{r}_i - \hat{r}_{i-1}|$, we obtain $|\hat{a}_i - a_i| \le \frac{e}{w}(N_i + N_{i-1}) =: \varepsilon_a$.

Step 2: Bounded perturbation of the statistic. Let $\hat{\chi}^2 = (\hat{a}_i - \hat{s}_i/n)^2 \cdot n^2 / [\hat{s}_i(n-1)]$ denote the estimated statistic. Conditioned on $\mathcal{G}$, the CMS errors in $\hat{a}_i$ and $\hat{s}_i$ introduce a bounded perturbation. Specifically, the numerator term satisfies $|\hat{a}_i - \hat{s}_i/n| \le |a_i - \bar{a}| + O(\varepsilon_a)$, and the denominator $\hat{s}_i$ differs from s_i by at most $O(n\varepsilon_a)$. For practical sketch configurations (width $w \ge 2^{12}$), $\varepsilon_a \ll \bar{a}$, so the perturbation $|\hat{\chi}^2 - \chi^2_{\text{true}}|$ is small relative to the detection threshold $\chi^2_{1-\alpha/2}$. In particular, we can choose w large enough that $\hat{\chi}^2 \le \chi^2_{1-\alpha/2}$ whenever $\chi^2_{\text{true}} \le \chi^2_{1-\alpha/2}$, i.e., the estimated statistic does not exceed the threshold unless the true statistic also approaches it.

Step 3: Union bound. Under H_0, the *true* statistic χ^2_{true} follows a chi-square distribution with 1 degree of freedom, so $P(\chi^2_{\text{true}} > \chi^2_{1-\alpha/2}) = \alpha/2$. Combining with the CMS failure probability:

$$\begin{aligned} P(\hat{\chi}^2 > \chi^2_{1-\alpha/2}) &\le P(\chi^2_{\text{true}} > \chi^2_{1-\alpha/2}) + P(\overline{\mathcal{G}}) \\ &\le \frac{\alpha}{2} + \frac{\alpha}{2} = \alpha \end{aligned} \tag{10}$$

□

Remark. The bound relies on a sufficient sketch capacity condition ($w \ge 2^{12}$, $d \ge \lceil \ln(2/\alpha) \rceil$), which ensures that CMS errors are small relative to the traffic signal. In our Tofino deployment, the default sketch configuration ($w = 4096$, $d = 3$) satisfies this condition for $\alpha = 0.05$, and the empirical false-positive rate in all experiments is below 2%.

A.2 Theoretical Bound on Stable-Period Suppression

BurstMon dynamically suppresses reports during stable periods. We derive the maximum unrecorded rate deviation during suppression and show how a single global threshold T provides rate-adaptive sensitivity.

For a timeslot to be classified as stable (i.e., filtered locally), its burst score must satisfy $\text{score}(f,i) \le T$. Substituting Equation 3 and rearranging (for $i > 1$ and $s_i > 0$):

$$a_i \le \frac{s_i + T \cdot \sqrt{s_i(i-1)}}{i} =: a_i^{\max} \tag{11}$$

This establishes an upper bound on the instantaneous rate deviation for any silently monitored timeslot. The actual flow rate r_i is constrained within $[r_{i-1} - a_i^{\max},\ r_{i-1} + a_i^{\max}]$, bounding the worst-case reconstruction error introduced by interpolation.

Asymptotic behavior. As the flow progresses ($i \to \infty$) and s_i grows proportionally, the average fluctuation converges to $\bar{a} = s_i/i$. The bound simplifies to:

$$a_i^{\max} \sim \bar{a} + T \cdot \sqrt{\bar{a}} \tag{12}$$

This reveals the adaptive nature of the threshold: the permissible unrecorded deviation scales with the square root of the

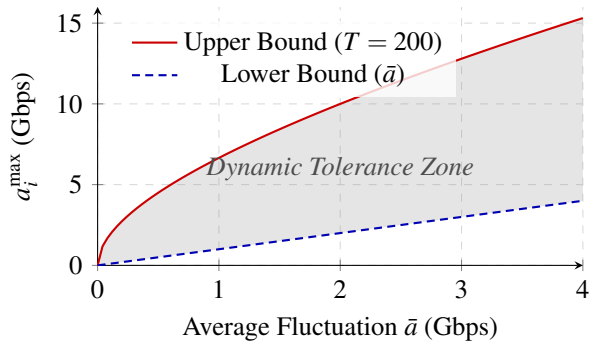


Figure 18: The χ^2-based dynamic tolerance zone with $T = 200$. The funnel-like shape suppresses benign noise for volatile flows while maintaining acute sensitivity for stable flows.

historical average fluctuation. Flows with high natural variability (large $\bar{a}$) are given wider tolerance, suppressing benign noise and preventing report storms. Conversely, stable high-rate flows (small $\bar{a}$) have tight tolerance, ensuring that even moderate deviations are flagged.

Finite-sample bound. For any finite timeslot i, the exact bound in Equation 11 holds without asymptotic approximation. When s_i is small (early in a flow's lifetime), the tolerance is also small, ensuring sensitivity during the initial phase.

Figure 18 visualizes this property: the funnel-like tolerance zone provides high relative tolerance for volatile flows while maintaining acute sensitivity for stable flows.

B Use Case 4: Proactive congestion signaling.

By increasing the threshold T, BurstMon filters out minor jitter and focuses exclusively on high-intensity bursts, thereby acting as a low-latency congestion warning engine. We evaluate this capability using the University of Wisconsin Data Center Measurement trace (UNI1), replayed at 50× speedup on a 10 Gbps link, following the experimental setting of ConQuest [7]. ConQuest is configured with its default congestion threshold $\tau = 0.8$ ms, while BurstMon operates at an 80 μs measurement granularity with $T = 512$; for BurstMon, we additionally account for a constant 100 μs uplink delay in the combined latency results.

Figure 19 plots the CDF of detection latency, defined as the elapsed time from the onset of queue buildup to the first detection report. Under this definition, BurstMon achieves a median latency of 190.3 μs and a maximum of 809.4 μs, whereas ConQuest has a median of 3204.6 μs and reaches 4773.8 μs in the tail. In ConQuest, τ constrains the queueing delay of the packet that triggers detection. However, because detection latency is measured from the first packet that experiences any nonzero queueing delay, a gradually growing queue can introduce a substantial interval before any packet first exceeds τ. Moreover, ConQuest reports only when such a packet is observed in the egress pipeline, and its snapshot-based design can introduce an additional phase delay before the packet becomes reportable. As a result, ConQuest is fundamentally reactive: it can signal congestion only after the queue has already grown sufficiently deep and the triggering packet has advanced to the reporting point. BurstMon, in contrast, reacts directly to abrupt rate changes in the data plane, allowing it to expose incipient congestion well before the queue reaches ConQuest's triggering condition.

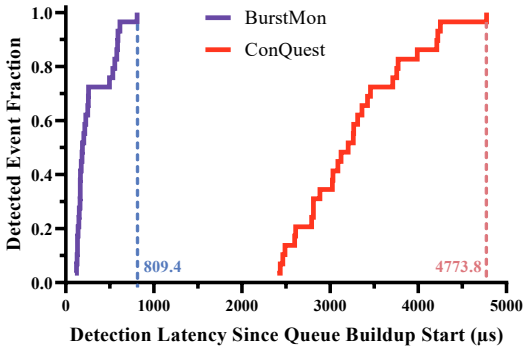


Figure 19: Detection latency of BurstMon vs. ConQuest.